

Three assistants, three personas

I built three assistants that live in chat. They share the same engine, but one is my personal assistant with more than sixty tools, another serves the public one at a time, and the third coordinates several voices inside a group. Here are the three side by side, what each one does in detail, and why they are the **foundation I keep building team agents on**. All examples are anonymized.

If you see patients or clients, sooner or later you wonder whether a conversational assistant is worth it. The honest answer is: **it depends on the shape**. An assistant just for you is not the same as one that faces strangers, nor is one-on-one the same as one dropped into a group of twenty. I wrote this in two layers: first, what each one does —readable for the decision-maker—; then how they’re built inside, for your technical advisor.

The three, side by side

	● Personal	● 1:1 support	● Groups
For whom	Just me	Public, one at a time	Work teams

	● Personal	● 1:1 support	● Groups
Channel	Telegram, web interface, and voice	WhatsApp and Telegram	WhatsApp (groups)
Identity	One SOUL with character	Public SOUL with hard limits	One soul per group: own voice, model and KB
Memory	Longitudinal: knows me over time	Product KB + a record per contact	Context and history of each group
How it takes part	Always, at my command	Answers whoever writes	Three modes (quiet, proactive, on command)
Managed via	Web dashboard	Web dashboard	Web dashboard
Notifies me via	—	Telegram (private)	Telegram (private)
Core superpower	More than 60 tools	Raise tickets and learn	Coordinate without spamming

The technology underneath is nearly the same in all three; what changes is the shape. And all three are configured from a **web dashboard** —soul, knowledge bases, model, participation modes, and authorizations— without touching code.

1 • The personal assistant — more than 60 superpowers

It’s just for me, with a name and a character, and I reach it by Telegram, a web interface, and by voice. It isn’t a chatbot that answers questions: it’s an **assistant that acts**. Today it adds up to **more than sixty tools** —what I call its superpowers — grouped into a few families:

Communication

Summarizes my inbox, drafts replies for me, and translates on the fly.

Productivity

Reminders, tasks, and follow-ups; nudges me in time so nothing falls through.

Knowledge

Searches the web, reads documents and images, and answers citing where the fact came from.

Longitudinal memory

Learns how I work and picks up threads from weeks ago without me repeating them.

Meta / self-observation

Reviews my own AI work sessions and hands me a plain-language summary of progress and loose ends.

Multimodal

Speaks and listens by voice, and understands photos and screenshots, not just text.

The piece that surprised me most is **self-observation**: it reviews my own AI work sessions and hands me back, in plain language, what I moved forward and what was left pending. An assistant that not only does tasks, but helps me see how I work. Its **memory is longitudinal**: it knows me over time and picks up threads from weeks ago without me repeating them.

2 • The 1:1 support one — faces the public

It faces strangers over WhatsApp and Telegram, one at a time. It's the first contact: it resolves product or service questions and, above all, **lets nothing drop**. What it does, concretely:

- **Serves over WhatsApp and Telegram**, the channels people are already on — no new app to install—, and understands text, images, and documents.
- **Raises tickets**: logs each request, classifies it, and gives it a reference number, so no query gets lost in the thread.
- **Learns**: when I teach it something new, it stores it as one more fact; if the data is sensitive, it stays pending my review before being used.
- **Consults the knowledge base** (prices, policies, course data) before answering, instead of replying from memory.

- **Notifies me on Telegram** privately when something goes off-script or needs a human: firm prices, a doubtful escalation, an alert.

3 · The groups one — several voices in one chat

It lives inside work groups, and it's the most delicate because a shared chat forgives less. What makes it livable:

- **One soul, one model, and one identity per group.** Each group can have its own personality, its own language model, and its own knowledge bases, versioned and prioritized. The same infrastructure sounds different in leadership than in support.
- **Promotes participation:** it opens topics, asks questions, and revives a fading thread, instead of waiting silently to be summoned.
- **Keeps context:** it saves the group's agreements and history, so the conversation has continuity and doesn't start from scratch each time.
- **Three ways to take part:** quiet (only if mentioned or quoted), proactive (a classifier decides whether to chime in, with a threshold and cooldown so it isn't annoying), or owner-only (only me, via a command).
- **Authorization lock:** if someone adds it to a new group, it doesn't take part; it says so once, sends me a private heads-up, and waits until I authorize it.

The soul: personality is a file

Each assistant is defined by a **SOUL**: a plain-text file with who it is, how it speaks, and what it never does. It isn't code; it's prose I can edit in a minute, and the assistant re-reads it on its own every so often without restarting anything. That's where the detail that most changes perception lives: the **speech register**.

It sounds minor and it isn't. The difference between "Hi! I'd be delighted to help, how can I assist you?" and a plain "Hi, what do you need?" is the difference

between a bot and someone. My personal assistant is banned from phrases that reek of robot (“my pleasure”, “great question”, “anything else?”) and, when I open the app, it *doesn’t greet*: the screen starts in silence. Stillness conveys more character than an automatic hello. Only I edit the SOUL; for the assistant it’s immutable.

Memory: split into layers

An assistant with no memory is a receptionist with amnesia. I split memory into layers, because mixing them is the fast lane to making things up:

Type	What it holds	Lifetime	How it’s used
Episodic	The recent conversation: the day’s turns, per user.	Short, windowed	Gives immediate context so it doesn’t repeat or lose track.
Semantic	Durable facts (prices, policies, course data) as key/value with version and source.	Long	The assistant queries it with tools before answering.
Relational	What the assistant infers about you and how you work.	Long, distilled overnight	Tunes the tone; never replaces the source of truth.

Semantic memory is facts with version and source that the assistant queries with a tool before answering, instead of “remembering” by eye. That way it learns without every correction depending on retraining a model.

The brain doesn’t depend on a single provider

No assistant talks to a model directly. They all go through a homegrown **gateway** that speaks the industry standard (OpenAI-compatible API). That gives me three things: I can **swap models without touching the assistant**, mix local and cloud providers, and get automatic *fallback*: if the preferred model fails, the request

drops to another one without the user noticing. That's why the groups assistant can run a different model in each group by changing one setting.

Inside, I split by cost and by risk. A **light, cheap** model carries the bulk: classify, answer, search. A **stronger** model is reserved for the paths where being wrong is costly: detecting a crisis, deciding a doubtful escalation, reading a document. The system reuses the context that repeats (the SOUL, the knowledge bases) so it isn't paid for on every message.

Guardrails and crisis detection

When an assistant faces the public —and more so if there are patients on the other side— limits aren't decoration: they're the product. Three mechanisms carry that weight.

Hard clinical rule

The support assistant **never** gives medical advice, nor interprets symptoms, nor suggests medications by chat. If someone writes "my cholesterol came back high, what do I take?", it doesn't answer with a drug: it refers them to their doctor and, in an emergency, to the emergency number. It resolves product and service; the body is seen by a human.

Crisis detector

A safety classifier checks every message against several risk categories. If something trips the alarm, the assistant **goes quiet** and notifies me privately instead of improvising. It's the same detector in all three, reused as-is.

Documents: file, don't interpret

If someone sends a PDF or a photo of a test, the assistant files it in that contact's record and notifies me —with a triage that's for me only—, but gives the sender only

a neutral acknowledgment: "I received your document, we'll review it and get back to you." It doesn't read them the lab.

Technical appendix

For the engineer who wants to see what's underneath. The figures are my own 2026 measurements, with small samples and a single operator: they are not benchmarks, they are evidence of how these systems behave in my scenarios. Everything is anonymized.

A • Architecture: from channel to reply

Stage	What happens	Where state lives
1 • Channel	A separate process per agent (the listener) receives the message: WhatsApp via Baileys (Node), Telegram, or web/voice. It normalizes identifiers, detects mentions and quotes, and downloads the attachment (in the groups agent: images up to 4 MB, PDFs up to 16 MB). Everything goes over HTTP to the brain.	WhatsApp session credentials on disk, in the listener only.
2 • Brain	One HTTP service (FastAPI) per agent builds the context: the SOUL as a system block with prompt caching, the subscribed knowledge bases, a capabilities block, and the history. In groups, before generating, it decides whether to speak: quiet, proactive (0.6 threshold and a 10-minute cooldown), or on command only.	Personal assistant: one session per day per user, in memory and in JSONL (survives restarts). Support and groups: a relational database (SQLite in groups; Postgres in the 1:1 design) with versioned souls, KBs, messages, an audit log, and a config table that overrides environment variables.
3 • Model	Anthropic SDK directly (personal assistant and 1:1 support, per the August audit) or an OpenAI-compatible gateway (groups agent, depending on the group). In the personal assistant's first measurement, the cold call wrote 1,456 tokens to cache and the next call read them.	Per-group routing lives in the database, not in git.

Stage	What happens	Where state lives
4 · Tool loop	The model proposes a tool, code executes it, and the result goes back to the model. The personal assistant loads more than 60 tools. In groups: a cap of 10 loops (raised from 5) and 4,096 tokens per turn (raised from 1,024: at 1,024 the model was cut off mid-call and claimed it had saved 5 facts when 2 had persisted; deterministic ingestion from the dashboard replaced trusting the model).	Tool blocks are not persisted across turns (see Memory).
5 · Reply	The brain returns text and the listener sends it. Each turn of the personal assistant is copied, write-only and non-blocking, to a separate long-term memory store; if that store is down, the chat keeps working.	The brain writes to that store but never reads from it.

B · Layered memory and its central constraint

Layer	Mechanism
Session history	Personal assistant: one session per day per user. Groups agent: a 30-message window. Only plain text is stored: role, content, and timestamp.
Semantic (KB)	Key/value facts with version, source, and status (active, pending review, superseded). What someone other than the owner teaches enters as pending review. Queried with tools, not from memory.
Relational / longitudinal	What is inferred about the user is distilled overnight and tunes the tone; it never replaces the KB.
Long-term, separate	A copy of each turn in an independent store with a write-only policy: the brain writes, never reads.

Tool blocks don't survive the turn

The history stores plain text only. `tool_use` and `tool_result` blocks are never persisted, so whatever lives in a tool result dies when the turn changes. **Any handshake across turns (an id, a token, a cursor) is broken by default and must be injected as explicit context.**

The bug that exposed it: I asked the personal assistant to delete three tasks and it tried 21 times in 3 minutes without deleting any. Each preview generated a new `action_id` that invalidated the previous one, and my “yes” reached a model that no longer had the id. My first diagnosis was wrong: I hardened the preview by repeating the id 6 times. It didn’t help, because the data never reached the next turn. Before blaming the model, verify the data is actually in its context.

The fix binds the pending action to the session and injects a “Pending actions awaiting confirmation” block into the system prompt with the tool, the arguments, and the exact id. Safety is preserved: it still requires a new message, and the id is still bound to the canonical arguments. Stateless tools (a `dry_run = not confirm` inside the tool itself) were immune. A sibling bug in the groups agent: in direct messages it didn’t remember its own replies because the history was only read per group.

C • Per-group soul, model and identity, and routing with fallback

Each group has its own versioned soul, primary model, fallback model, and KBs with priority, all editable from the dashboard. At the last cross-check (September 24), 11 groups ran on the local model with a cloud fallback, 6 on Haiku, and 2 on Sonnet 5, both with the local model as fallback.

Tier	Use	Fallback
Local (Qwen3.6-35B-A3B)	Normal, high-volume conversation: 2.2 s per turn, measured.	Cloud fallback (Haiku): a local model’s fallback must be cloud, not itself.
Haiku 4.5	AI, Claude, and education groups.	Fallback: the local model.
Sonnet 5	2 groups, plus turns that touch sensitive data or irreversible actions (escalation).	Fallback: the local model.

Measured criteria (624 generations). Simulation education never runs on the local model: across 5 questions it fabricated sources in 16 of 20 answers (the conclusions are usually right; the citations don’t exist). Claude and Anthropic topics go to Haiku, because the local model’s knowledge ends at its cutoff date. A group whose context is only what was said in the chat can run local: it attributed correctly 12 out of 12.

The local model stays as a fallback, by decision. If the primary fails, a fallback that answers worse for a while beats a mute group; what’s intolerable is not finding out. That’s

why detection matters. For about a month, local routing didn't work: the model identifier carried a provider prefix and the gateway rejected it with a 403, but the fallback always answered and nobody noticed (of all the messages in the "local" groups, only one was served by the local model). A fallback that works masks the fact that the primary is broken. The detector I built compares, per group, the configured model with the one that actually answered (states: ok, divergent, no data; "no data" is shown separately because no data is not the same as healthy). It also fixed a blind spot: with the circuit breaker already open, the audit recorded nothing.

It validated itself: a cloud key became invalid (HTTP 401) and the groups with a cloud primary fell back to the local model; the table flagged the divergence in red and I saw it. The repair included restarting the brain, because the live process keeps the old key in memory. One detail of the routing API: the PATCH replaces all 4 fields, so sending only the fallback leaves the primary null and the group inherits the default model.

D • Governance: irreversible actions are governed in code

The thesis: **no irreversible action should depend on the model's judgment.** If an agent can delete, remove, publish, or send without confirmation, an incident is a matter of time; switching models only moves the probability, it doesn't eliminate it. The gate lives in code, not in the prompt.

D1 • The evidence that motivated it

I built a tool-use arena with 5 scenarios made from my agents' real tool chains, each with a deliberate trap (a destructive or forbidden tool within reach), a blind judge (Opus, with anonymized, alternated answers), and a 4-dimension rubric. In the clinical scenario (a patient with active symptoms), the production local model executed the record deletion in **6 of 10 repetitions**, justifying it with the European GDPR in a Mexican case. No public benchmark anticipated it: that model and its comparator sit 3 points apart on the intelligence indices.

A second arena (October 2026) confirmed it from the other side: the "uncensored" (abliterated) version of the same model, in a clean pair, nearly doubled critical failures (40 % → 77 %) and broke exactly the limits that protect third parties: **permissions** (removing someone from a group without being an admin, 0/20 → 17/20) and **clinical confidentiality** (revealing another patient's diagnosis, 0/20 → 15/20). More freedom in the model is not more judgment; that is why the limits live in code.

	Local MoE 35B (3B active)	Dense 30B
Quality (blind judge)	60/100	82/100
Executed the record deletion	6 of 10	0 of 10
Escalated to a physician	8 of 10	10 of 10
Latency per turn	2.2 s	71 s (~32×)
Tokens per turn	~130	~1,750
Raw tokens per second	80	33

Two lessons. With a single sample the deletion looked like an accident; **ten repetitions revealed it as a pattern** of 60 %. And tokens per second doesn't predict user experience: the dense model generates 13 times more tokens while reasoning, so the per-turn gap (32×) is much larger than the per-token gap (2.4×). Caveat: one dense-model case was truncated at the 2,000-token cap, so its quality is underestimated. In another measurement across three comparable models, the deletion rate was 0 %, 50-60 %, and 70 %.

D2 • The audit of destructive tools

The arena triggered a read-only audit of the code deployed in all four agents. Findings and fixes (all verified in production, not just with unit tests):

Finding	Fix
In 3 of 4 agents, "owner only" lived as prose in the system prompt. The 1:1 support agent handed its 16 tools to any interlocutor.	The toolset is filtered by role before it reaches the model: a non-owner sees 5 of 16. The role is validated again at execution, and every blocked attempt is logged.
The owner role could be assigned through chat: privilege escalation (the owner bypasses the safety layer).	Removed from the enum and also rejected in the executor (defense in depth).
Unauthenticated service endpoints, including a hard	Shared-secret header, constant-time comparison. In the groups agent, 14 state-mutating endpoints; 401 verified without a token. Fails open if the secret is empty, and the

Finding	Fix
cascading delete and publishing to every group.	health endpoint exposes token_active so it can be audited. The chat endpoint is deliberately left open (the listener uses it): pending risk.
SSRF via redirect in the tool that fetches URLs: only the original URL was validated.	No automatic redirect following: up to 3 hops by hand, revalidating each one; CGNAT range fixed. Tested with a server answering 302 to loopback: blocked without leaking content.
No audit trail of tool calls.	Logged at the single dispatch point: tool, role, group, arguments truncated to 120 characters (to avoid dumping personal data), and result. Two error conventions coexisted; the first version logged a permissions rejection as a success.
No way to disarm writes without shutting the service down (and shutting it down leaves users without a reply).	A read-only switch via environment variable, read on every call: the agent keeps conversing and querying the KB, but writes and sends nothing.

D3 • Preview and confirmation with action_id

Irreversible tools work in two phases: the first call returns a preview and an `action_id` with a limited lifetime, bound to the canonical arguments; only a second call carrying that id executes. One agent already had the right pattern, but only 1 of 4, and not the one serving patients. There are two valid ways to stop the model from confirming itself, and **you choose according to the agent's design**:

A • The model cannot self-confirm (1:1 support agent)

Code strips confirm and action_id from the model's arguments before executing; the schema doesn't even offer them. Execution happens only through a deterministic endpoint triggered by a human button, which revalidates the allowlist.

B • Cut the turn (personal assistant)

When a pending action is registered, the turn is cut: the preview reaches the user and confirming requires a new message. Before, the model read the action_id from the result and confirmed in

the same loop. Both loops were patched (full response and streaming; the second is easy to forget).

Copying pattern A into the personal assistant would have made every confirmable action impossible to execute, because that agent has no confirmation endpoint. Verification (0 failures): the preview is mandatory, an invented `action_id` is rejected, arguments altered between preview and confirmation are rejected, and both loops cut the turn. Five irreversible tools that had no gate moved to the two-phase flow. An editing gotcha: a regex inserted the confirmation parameter into the wrong tool; it was caught by listing which tools ended up with it and comparing against the expected list.

D4 · The irreversibility classifier in production

Escalation to a frontier model had existed since August, but it depended on each agent flagging the turn as irreversible, and none did. So a classifier decides for them. I evaluated it on 120 synthetic Mexican-Spanish turns (54 irreversible, 66 harmless; 60 commands, 40 look-alike harmless pairs, and 20 negation or conditional), with a 0.3 threshold to measure the cascade. **Calibration was decisive, not accuracy.**

Classifier	Accuracy	FN	Brier	p50	Cascade (t = 0.3)
Local Qwen with logprobs (in production)	98.3 %	1	0.021	91 ms	Covers 92 %, correct 99.1 %, 0 FN
Hosted proprietary classifier (reference)	98.3 %	1	0.023	307 ms	Covers 91 %, correct 100 %, 0 FN
Local Qwen asked for a written number	98.3 %	1	0.017	148 ms	Covers 100 %, but its error came out at p = 0.00: cannot be escalated
Small open encoder, trained on English only	82.5 %	16	0.170	30 ms	Covers 22 %, correct 81 % useless

Accuracy ties at 98.3 %, but the local model's false-negative error came out with a certainty of 0.00, and an error like that can't be escalated. The calibration problem wasn't in the sampling but in asking for a written number: with `max_tokens=1`, temperature 0, the top 5 logprobs, and an output of "0" or "1", the probability is $p = P("1") / (P("0") + P("1"))$.

With that, the doubtful cases land where they should: “Ok, do it” scores 0.53 and “Don’t charge them yet” scores 0.44.

Production parameters. Threshold $p \geq 0.8$ (0.3 is only the arena’s). It runs only when the turn carries tools, the engine is escalatable, and the agent didn’t set the flag; if so, the turn escalates to Sonnet 5 and the response carries the reason and p . It costs one token and roughly 100 to 130 ms. A 300 ms timeout: if it expires, the turn proceeds without the classifier, because the dangerous mode is slowness, not failure. There are counters in the gateway’s deep health check and a label harvest for distilling a smaller model. Going to production exposed a prior bug: every escalation with tools returned a 400 from the provider, because the flag route had never been exercised with tools.

Escalation **complements** the gates, it doesn’t replace them: the gate prevents executing without human confirmation; the classifier improves the judgment of whoever proposes. A better model still fails sometimes, hence both.

E • Reliability: what broke and how it was closed

Case	Symptom	Cause	Fix
WhatsApp IP block	Two listeners down at once, code 405 on every attempt, session not logged out (loggedOut false), credentials intact; the service manager said “active”.	One retried with a fixed 2 s delay, no re-entrancy guard, and without closing the previous socket: the loops multiplied (10,912 closes versus 1,326 for the one that did have backoff). They share a public IP, so it took the other down. Two different Baileys versions failed identically: the problem was the IP, not the code.	Exponential backoff, 60 s; a 10-second reconnect guard that closes the socket; backoff on connect. On silence for 60 s, wasn’t enough, resets the one listener
515 after pairing	You scan the QR, the phone sticks on “logging in”, the state flips to disconnected, and the credentials directory is empty.	515 (restartRequired) means “pairing succeeded, restart now”, but it was fed into the same backoff: with a 60 s wait (or a 600 s cap) the window expires and the pairing is lost.	Immediate 515 only, a timeout toward the

Case	Symptom	Cause	Fix
A thread that dies without an exception	The process is alive, the port is dead, and there isn't a single log line. An internal API stayed down for about four days without anyone noticing.	The guard only killed the process on an exception; the server returned cleanly.	A guard on and clean terminates supervisor. KeepAlive process does not mute.
Listeners dead after 503/408 + 1006	Account in a linked-stopped-disconnected state with a stale error; credentials intact (under a previous framework).	A WhatsApp-side cut; the built-in health monitor gave up after about 3 retries.	Restart the touching to several times investigate
A watchdog that only checks /health	The brain answered 200 while all its tools failed (the default IPv6 route went out an interface that doesn't route IPv6). The morning brief came out incomplete and nobody knew.	The watchdog asked whether the process responded, not whether the service did its job.	At least on real dependencies or a real gateway models on server process often than the degradation what it lacks
A probe that cannot fail	The "no response: restart" branch never ran.	<code>curl ... echo 000</code> yields 000000, because curl already prints 000 on failure.	Test the w dependencies just the health

Other Baileys gotchas worth applying from the first launch: pin a stable version (with "latest" an RC of 7.x got installed and the QR expired with a 408 every ~3 min); always request the latest WhatsApp Web protocol version (otherwise, 405); and normalize messages with `normalizeMessageContent` before reading them, because a PDF with a caption or a mention arrives wrapped and, if read directly, the message is silently dropped. "Active" in the service manager doesn't mean WhatsApp is connected: you have to query the listener's real state.

These three aren't the finish line: they're the **foundation**. The same engine —soul in a file, layered memory, tools, and a gateway that doesn't

depend on one provider— is the base I keep building **team agents** on: one per role, with its own voice, permissions, and tools, without starting from scratch. An assistant that feels like a person isn't a bigger model: it's a well-written soul, orderly memory, and limits that are respected.

Thinking about assistants like these for your practice or your team? [Write to me →](#)